

Raporty własne w InsERT nexo PRO

Spis treści

Tworzenie raportów własnych w systemie InsERT nexo.....	2
Dodawanie raportu własnego - sekcja PODSTAWOWE	2
Dodawanie raportu własnego - sekcja PARAMETRY	3
Parametr tekstowy	3
Parametr datowy	4
Parametr zakresu dat	7
Parametr listowy i listowy z wielowymybozem	8
Parametr liczbowy	10
Parametr listowy bazodanowy	12
Parametry globalne	14
Dodawanie raportu własnego - sekcja ŹRÓDŁO DANYCH	15
Parametry globalne	16
Standardowe (proste) pola własne	17
Zaawansowane pola własne w wersji 2	17
Dodawanie raportu własnego - sekcja KOLUMNY	19
Kolumna tekstowa	21
Kolumna datowa.....	21
Kolumna liczbowa	21
Kolumna logiczna	22
Dodawanie raportu własnego - sekcja PODGLĄD	23
Eksportowanie i importowanie	23
Uruchamianie raportów własnych LINQ oraz jego edycja i podgląd.	24
Testowanie raportów własnych w LINQPadzie	25

Tworzenie raportów własnych w systemie InsERT nexo

Definiowanie raportów własnych LINQ jest dostępne w wersji nexo PRO. Aby zdefiniować raport, konieczne jest napisanie zapytania w języku LINQ (*LINQ to Entities*), które będzie wykorzystywać model danych systemu InsERT nexo. W celu utworzenia raportu własnego należy otworzyć **moduł Raporty** (Ctrl+spacja, mnemonik: RP) i wykorzystać polecenie **Dodaj** (Insert) w górnym menu. Otwarte zostanie okno edycji raportu własnego. Okno to składa się z 5 sekcji: podstawowe, parametry, źródło danych, kolumny oraz podgląd. Do dodawania nowych raportów własnych LINQ użytkownik musi mieć nadane uprawnienie (Raporty własne - edycja).

Narzędzie pomocnicze do tworzenia zapytań w LINQ opisane jest w rozdziale *Testowanie raportów własnych w LINQPadzie*.

Dodawanie raportu własnego - sekcja PODSTAWOWE

Sekcja zawiera elementy opisowe i służące odróżnieniu go na liście w module Raporty.

PODSTAWOWE	
Nazwa:	Przykładowe zestawienie asortymentu
Główny element:	Asortyment
Opis:	To jest przykładowe zestawienie własne w Subieckie nexo.
Grupa:	Asortyment
Widoczny w:	
☒ Subiekt	
☒ Rachmistrz	
☒ Rewizor	
☒ Gratyfikant	
☒ Gestor	
☒ Biuro	

rys. 1. Sekcja Podstawowe formularza edycji raportu własnego LINQ.

- **Nazwa** - nazwa raportu, widoczna na liście w module raportów. Każdy raport musi mieć nadaną nazwę, nie dłuższą jednak niż 100 znaków.
- **Główny element** – wskazuje na dziedzinę raportu, czy będzie to zapytanie dotyczące asortymentów, klientów, dokumentów czy innych obiektów dostępnych w systemie InsERT nexo. Są to tzw. encje modelu danych. Wybór dowolnej opcji wypełni zawarte w sekcji ŹRÓDŁO DANYCH pole **Zapytanie** domyślną wartością (o ile pole to nie będzie miało wprowadzonej wcześniej zawartości). Określenie głównego elementu raportu powoduje także udostępnienie w wynikach raportu komend przeznaczonych dla wybranego typu obiektu. Tych samych, które są dostępne w jego macierzystym module.
- **Opis** - krótka informacja dla użytkownika o przeznaczeniu raportu. Pole obligatoryjne; maksymalna długość opisu to 255 znaków. Jego zawartość jest widoczna w podglądzie modułu Raporty po wybraniu raportu na liście.
- **Grupa** - określa przyporządkowanie tworzonego raportu do jednej z grup widocznych na liście w module Raporty. Po rozwinięciu dostępna jest lista aktualnie zdefiniowanych grup

raportów; możliwe jest jednak utworzenie nowej grupy poprzez wpisanie jej nazwy. Maksymalna długość nazwy grupy to 64 znaki, brak wartości nie jest dopuszczalny. Grupa zostanie dodana wraz z zapisaniem tworzonego raportu.

- **Widoczny w** – wskazuje produkty z linii InsERT nexo, w których tworzony raport własny będzie widoczny oraz możliwy do uruchomienia. Domyślnie wybrany jest produkt, w którym tworzony jest raport.

Dodawanie raportu własnego - sekcja PARAMETRY

Sekcja ta służy zdefiniowaniu **dynamicznie obsługiwanych parametrów** raportu własnego. Parametry te mogą być pomocne podczas definiowania zapytania w procesie tworzenia **źródła danych** raportu własnego a następnie wykorzystywane przy uruchamianiu raportu.

PARAMETRY			
Lp	Nazwa	Etykieta	Typ
1	początekNazwy	Nazwa rozpoczyna się od	Tekst

rys. 2. Sekcja Parametry formularza edycji raportu własnego LINQ.

Każdy parametr musi mieć nadaną unikalną nazwę zgodną z zasadami języka C# (<http://msdn.microsoft.com/en-us/library/aa664670.aspx>). Nie mogą to być słowa kluczowe języka C#. Zawartość pola **Etykieta** będzie pokazywana użytkownikowi obok elementu interfejsu służącego do wybrania wartości parametru nad listą z wynikiem zapytania (po uruchomieniu raportu w module). Typ natomiast precyzuje rodzaj parametru – obecnie dostępny jest parametr tekstowy, datowy, liczbowy, listowy, listowy z wielowymianem oraz listowy bazodanowy.

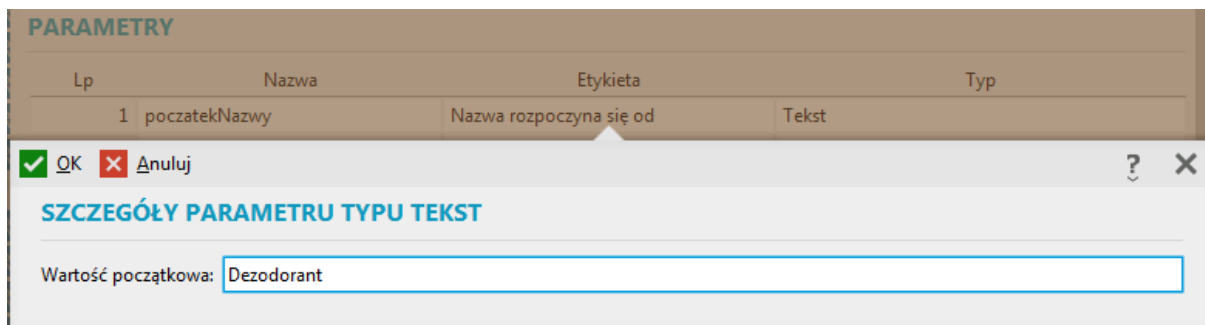
Parametry będą pokazywane w raporcie zgodnie z kolejnością ich wprowadzenia na liście. Kolejność tę można zmienić, korzystając z opcji menu kontekstowego: przenieś na początek (Ctrl+Shift+strzałka w górę), przenieś wyżej (Ctrl+strzałka w górę), przenieś niżej (Ctrl+strzałka w dół), przenieś na koniec (Ctrl+Shift+strzałka w dół). Dla każdego z parametrów można ustawić pewne dodatkowe właściwości, dostępne po wybraniu z menu kontekstowego opcji Szczegóły (Ctrl+E).

W sekcji **Źródło danych** w zapytaniu LINQ należy wykorzystywać parametry posługując się ich **nazwą**.

Parametr tekstowy

Parametr ten służy do pobrania tekstu od użytkownika. Na interfejsie użytkownika wyświetlone zostanie standardowe pole tekstowe. Dodatkowe właściwości parametru zawierają tylko wartość początkową, która nie jest obowiązkowa. Pozostawione puste pole

oznacza, że początkową wartością tego parametru jest pusty ciąg. Przy uruchomieniu raportu, ta specjalna wartość parametru jest dostępna pod opcją (*pusty*).



Lp	Nazwa	Etykieta	Typ
1	początekNazwy	Nazwa rozpoczyna się od	Tekst

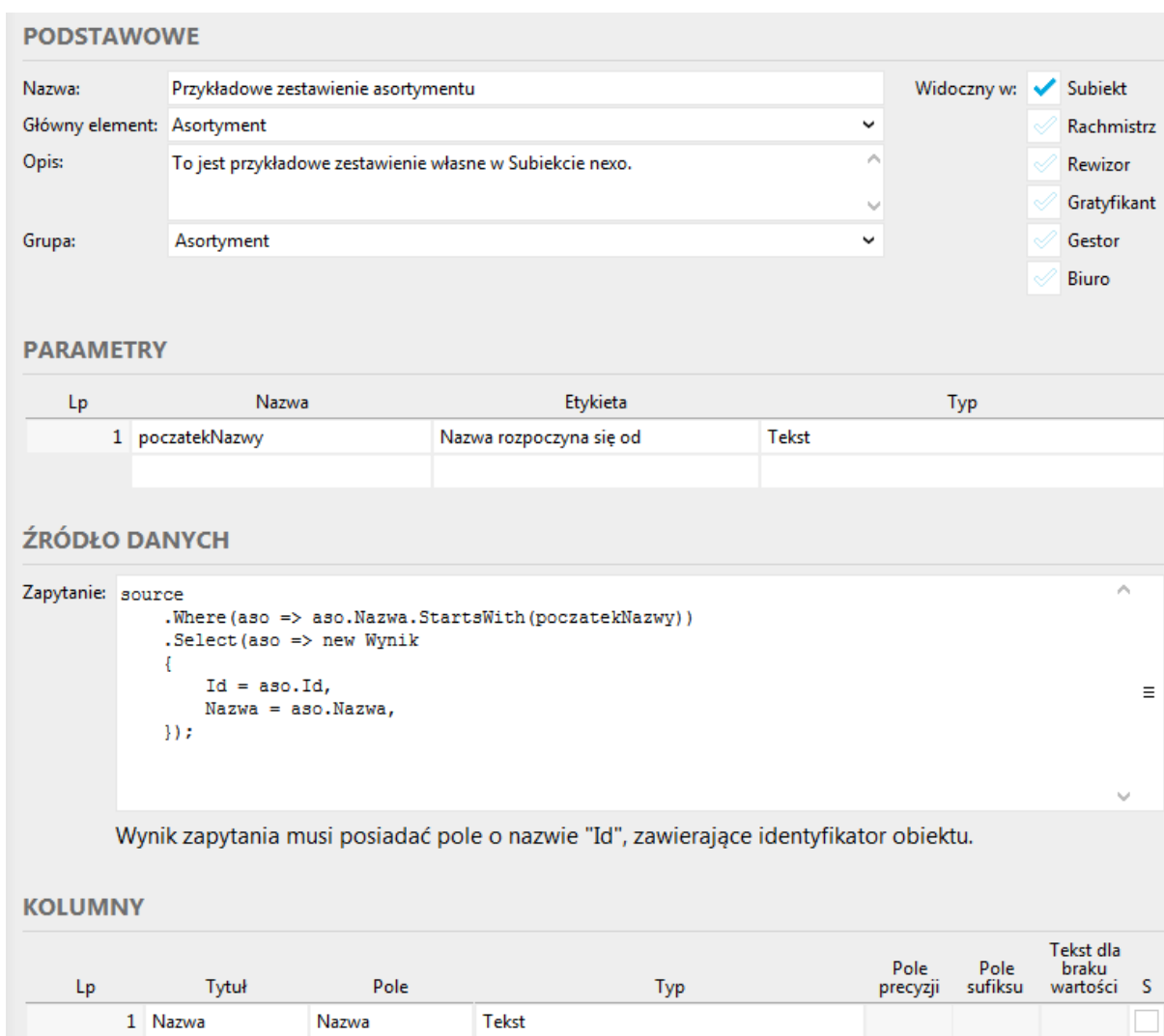
OK Anuluj

SZCZEGÓŁY PARAMETRU TYPU TEKST

Wartość początkowa:

rys. 3. Szczegóły parametru tekstowego.

Przykład raportu własnego z użyciem parametru tekstowego:



PODSTAWOWE

Nazwa: Przykładowe zestawienie asortymentu

Główny element: Asortyment

Opis: To jest przykładowe zestawienie własne w Subiekcie nexo.

Grupa: Asortyment

Widoczny w: ☒ Subiekt ☒ Rachmistrz ☒ Rewizor ☒ Gratyfikant ☒ Gestor ☒ Biuro

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	początekNazwy	Nazwa rozpoczyna się od	Tekst

ŹRÓDŁO DANYCH

Zapytanie:

```
source
    .Where(aso => aso.Nazwa.StartsWith(początekNazwy))
    .Select(aso => new Wynik
    {
        Id = aso.Id,
        Nazwa = aso.Nazwa,
    });
```

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

KOLUMNY

Lp	Tytuł	Pole	Typ	Pole precyzji	Pole sufiksu	Tekst dla braku wartości	S
1	Nazwa	Nazwa	Tekst				☐

Parametr datowy

Parametr tego rodzaju służy do wybrania daty. Na interfejsie użytkownika wyświetlony zostanie rozwijalny kalendarz do wprowadzenia wartości parametru. Dodatkowe właściwości

parametru pozwalają na wybór domyślnej wartości parametru oraz dopuszczenie braku wartości.

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	data	Data wystawienia	Data

☒ OK
 ☐ Anuluj

SZCZEGÓŁY PARAMETRU TYPU DATA

Wartość początkowa:

Dopuszczaj brak wartości: ☒

rys. 4. Szczegóły parametru datowego.

Przykład raportu własnego z parametrem datowym:

PODSTAWOWE

Nazwa:
 Widoczny w: ☒ Subjekt

Główny element:
☒ Rachmistrz

Opis:
☒ Rewizor

Grupa:
☒ Gratyfikant

☒ Gestor

☒ Biuro

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	data	Data sprzedaży	Data

ŹRÓDŁO DANYCH

Zapytanie:

```

source
.Where(dok => dok.DataSprzedazy == data)
.Select((DokumentDS dokumentDS) => new Wynik
{
    Id = dokumentDS.Id,
    Wartosc = dokumentDS.Wartosc.BruttoPoRabacie,
    Data = dokumentDS.DataSprzedazy,
    Numer = dokumentDS.NumerWewnetrzny.PelnaSygnatura
});
    
```

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

KOLUMNY

Lp	Tytuł	Pole	Typ	Pole precyzji	Pole sufiksu	Tekst dla braku wartości	S
1	Data	Data	Data i czas				☐
2	Numer	Numer	Tekst				☐
3	Wartość	Wartosc	Liczba				☐

Dopuszczenie braku wartości parametru i wybór opcji (*brak*) przy uruchamianiu skutkują przekazaniem do zapytania specjalnej wartości tego parametru. Należy to odpowiednio uwzględnić i wykorzystać w zapytaniu poprzez dodatkowe przyrównanie z pseudo-wartością *null*. Poniżej przedstawiono przykład zmodyfikowany w taki sposób, że przy ustawieniu daty na wartość (*brak*) pokażą się wszystkie dokumenty, bez względu na wartość pola *DataSprzedazy*.

Przykład wykorzystania braku wartości parametru datowego:

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	data	Data sprzedaży lub (brak) gdy dowolna	Data

ŹRÓDŁO DANYCH

Zapytanie:

```

source
.Where(dok => data == null || dok.DataSprzedazy == data)
.Select((DokumentDS dokumentDS) => new Wynik
{
    Id = dokumentDS.Id,
    Wartosc = dokumentDS.Wartosc.BruttoPoRabacie,
    Data = dokumentDS.DataSprzedazy,
    Numer = dokumentDS.NumerWewnetrzny.PelnaSygnatura
});

```

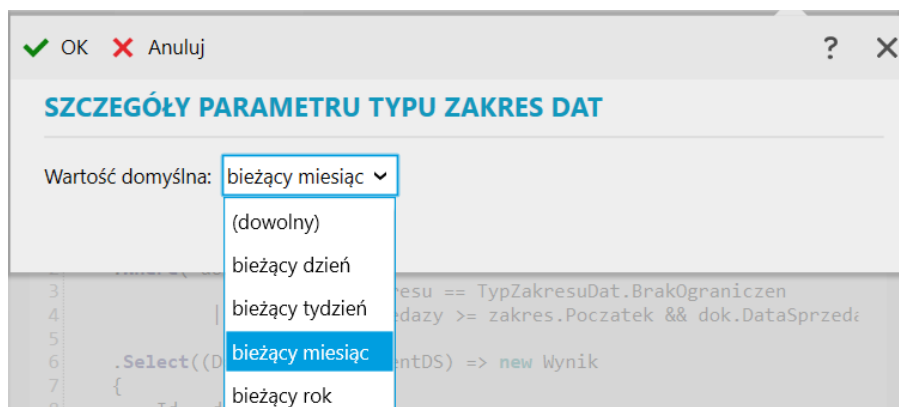
Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

KOLUMNY

Lp	Tytuł	Pole	Typ	Pole precyzji	Pole sufiksu	Tekst dla braku wartości	S
1	Data	Data	Data i czas				☐
2	Numer	Numer	Tekst				☐
3	Wartość	Wartosc	Liczba				☐

Parametr zakresu dat

Parametr ułatwia wybór zakresu dat. Definiując parametr, poza standardowymi cechami parametru, takimi jak Tytuł, Pole i Typ możemy wskazać domyślną wartość.



rys. 5. Szczegóły parametru typu Zakres dat

Przykład raportu, z parametrem typu Zakres dat.

LQ

_ZD
Dokumenty sprzedaży

PODSTAWOWE PARAMETRY

Lp	Nazwa	Etykieta	Typ	Opis parametru
1	zakres	Data dokumentu	Zakres dat	Wartość domyślna: bieżący...

ŹRÓDŁO DANYCH

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

```

1 source
2   .Where( (DokumentDS dok) =>
3       zakres.TypZakresu == TypZakresuDat.BrakOgraniczen
4       || (dok.DataSprzedazy >= zakres.Poczatek && dok.DataSprzedazy <= zakres.Koniec))
5
6   .Select((DokumentDS dokumentDS) => new Wynik
7   {
8       Id = dokumentDS.Id,
9       Wartosc = dokumentDS.Wartosc.BruttoPoRabacie,
10      Numer = dokumentDS.NumerWewnetrzny.PelnaSygnatura,
11      DataSprzedazy = dokumentDS.DataSprzedazy
12  });

```

Definiując parametry typu Zakres dat w definicji raportu mamy do dyspozycji zmienną typu **ZakresDat**. W powyższym przykładzie zmienna ta nazywa się **zakres**.

Typ ZakresDat ma następujące właściwości:

```
public TypZakresuDat TypZakresu { get; set; }
```

```
public DateTime? Początek { get; set; }
public DateTime? Koniec { get; set; }
```

Opis wartości TypZakresuDat

Wartość	Opis w kontrolce	Uwagi
BrakOgraniczen	(dowolny)	Właściwości Początek i Koniec nie są ustawione (Początek.HasValue i Koniec.HasValue mają wartość false)
BiezacyDzien	bieżący dzień	
BiezacyTydzen	bieżący tydzień	
BiezacyMiesiac	bieżący miesiąc	
BiezacyRok	bieżący rok	
BiezacyOkresObrachunkowy	bieżący okres obrotowy	Dostępne w Rachmistrzu i w Rewizorze
WybranyOkres	dowolny okres	Korzysta z dat, podanych w dodatkowym oknie

Dla typów innych niż BrakOgraniczen właściwość Początek zawiera datę bez części godzinowej (początek dnia), a właściwość Koniec zawiera datę z maksymalną wartością części godzinowej (koniec dnia).

Np. dla BiezacyRok będą to wartości:

Początek: 1-01-2020 00:00:00, Koniec: 31-12-2020 23:59:59.

Parametr listowy i listowy z wielowymyślnym

Parametry listowe umożliwiają użytkownikowi wybór jednego (lub kilku) elementów spośród listy dostępnych. W przypadku parametru listowego niezbędne jest otwarcie sekcji **Szczegóły** w celu zdefiniowania listy dopuszczalnych elementów. Sekcja ta umożliwia również wskazanie początkowo wybranego elementu oraz na dopuszczenie braku wyboru.

The screenshot shows a window titled 'PARAMETRY' with a tab 'Szczegóły parametru typu LISTA'. Inside, there's a table with columns 'Lp', 'Nazwa', 'Etykieta', and 'Typ'. Below the table, there's a section 'Lista elementów:' with a list of months: 1 styczeń, 2 luty, 3 marzec, 4 kwiecień, 5 maj, 6 czerwiec, 7 lipiec, 8 sierpień. At the bottom, there are two checkboxes: 'Początkowy wybór:' with a dropdown set to '(brak)', and 'Dopuszczaj brak wyboru:' which is checked.

rys. 6. Szczegóły parametru listowego.

Każdy z elementów listy musi posiadać wartość liczbową, dzięki której możliwe jest odróżnienie od siebie wybranych przez użytkownika elementów. Sytuacja gdy użytkownik wybierze element specjalny (*brak*), tak jak w przypadku parametru datowego, może być obsługana przez jej odpowiednie zinterpretowanie w samym zapytaniu. Należy to uwzględnić przez dodatkowy warunek wykorzystujący pseudo-wartość *null* tego elementu.

Przykład raportu własnego z parametrem listowym:

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	miesiac	Miesiąc	Lista

ŹRÓDŁO DANYCH

Zapytanie:

```

source
.Where(dok => miesiac == null || dok.DataSprzedazy.Month == miesiac)
.Select((DokumentDS dokumentDS) => new Wynik
{
    Id = dokumentDS.Id,
    Wartosc = dokumentDS.Wartosc.BruttoPoRabacie,
    Data = dokumentDS.DataSprzedazy,
    Numer = dokumentDS.NumerWewnetrzny.PelnaSygnatura
});
        
```

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

KOLUMNY

Lp	Tytuł	Pole	Typ	Pole precyzji	Pole sufiksu	Tekst dla braku wartości	S
1	Data	Data	Data i czas				☐
2	Numer	Numer	Tekst				☐
3	Wartość	Wartosc	Liczba				☐

Parametr listowy z wielowymierzem, wykorzystywany w zapytaniu, udostępnia kolekcję wartości wybranych z listy elementów. Specjalnym elementem, który można dołączyć do listy wyboru, jest (*brak*) z wartością *null*. Poza tym jest traktowany jak każdy inny element z listy. Przy uruchomieniu, oprócz zdefiniowanych elementów, lista z wielowymierzem udostępnia dwa dodatkowe. Pierwszy z nich to (*wybrane*), za którym kryje się możliwość wyboru wielu elementów listy. Drugi element to (*dowolny*), którego wybranie jest tożsame z wyborem wszystkich elementów listy. W przypadku, gdy użytkownik nie wybierze żadnego elementu, kryjąca się pod nazwą parametru kolekcja wykorzystywana w zapytaniu, jest pusta.

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	miesiace	Miesiące	Lista wielokrotnego wyboru

OK Anuluj Lista

SZCZEGÓŁY PARAMETRU LISTOWEGO Z WIELOWYBOREM

Lista elementów:

Wartość	Nazwa
1	styczeń
2	luty
3	marzec
4	kwiecień
5	maj
6	czerwiec

Początkowy wybór:

- ☒ (brak)
- ☒ styczeń
- ☒ luty
- ☒ marzec
- ☒ kwiecień
- ☒ maj
- ☒ czerwiec

2 zaznaczone

Filtruj...

Widoczne: wszystkie

Zaznacz widoczne

Odnznacz widoczne

Dopuszczaj element (brak): ☒

rys. 7. Szczegóły parametru listowego z wielowymybarem.

Przykład wykorzystania listy z wielowymybarem:

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	typy	Typ rezerwacji	Lista wielokrotnego wyboru

ŹRÓDŁO DANYCH

Zapytanie:

```
source
    .Where(dok => typy.Contains(dok.TypRezerwacji))
    .Select((DokumentZK dokument) => new Wynik
    {
        Id = dokument.Id,
        Data = dokument.DataSprzedazy,
        Numer = dokument.NumerWewnetrzny.PelnaSygnatura,
        Wartosc = dokument.Wartosc.BruttoPoRabacie,
    });
```

Parametr liczbowy

Parametr tego rodzaju służy do wprowadzenia wartości liczbowej. Właściwości parametru pozwalają na ustalenie początkowej wartości parametru oraz dopuszczenie braku wartości. Precyzja to liczba całkowita, nieujemna. Sterowanie jej wartością pozwala określać, czy parametr jest całkowitoliczbowy (precyzja 0) czy zmiennoprzecinkowy (precyzja > 0).

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	wartosc	Wartość minimalna	Liczba

OK

Anuluj

?

×

SZCZEGÓŁY PARAMETRU TYPU LICZBA

Wartość początkowa:

10

Precyzja:

0

Dopuszczaj brak wartości:

☒

rys. 8. Szczegóły parametru liczbowego.

Przykład raportu własnego z parametrem liczbowym:

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	wartosc	Wartość minimalna	Liczba

ŹRÓDŁO DANYCH

Zapytanie:

```

source
.Where(dok => wartosc == null || dok.Wartosc.BruttoPoRabacie >= wartosc)
.Select((DokumentDS dokumentDS) => new Wynik
{
    Id = dokumentDS.Id,
    Wartosc = wartosc, //dokumentDS.Wartosc.BruttoPoRabacie,
    Data = dokumentDS.DataSprzedazy,
    Numer = dokumentDS.NumerWewnetrzny.PelnaSygnatura
});

```

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

KOLUMNY

Lp	Tytuł	Pole	Typ	Pole precyzji	Pole sufiksu	Tekst dla braku wartości	S
1	Data	Data	Data i czas				☐
2	Numer	Numer	Tekst				☐
3	Wartość	Wartosc	Liczba				☐

Zdefiniowany przez nas parametr ma zaznaczoną opcję **Dopuszczaj brak wartości**. Aby prawidłowo obsłużyć sytuację, gdy użytkownik wybierze (*brak*), zapytanie zawiera dodatkowy warunek uwzględniający ten specyficzny przypadek. W naszym przykładzie ta specjalna wartość *null* spowoduje wyłączenie filtrowania wg wartości brutto dokumentu.

Parametr listowy bazodanowy

Parametr listowy bazodanowy, podobnie jak zwykły parametr listowy, umożliwia użytkownikowi wybór jednego elementu z określonej listy. W przypadku tego parametru lista dostępnych elementów nie jest jednak stała, tylko definiowana za pomocą zapytania bazodanowego. W przypadku parametru tego typu niezbędne jest otwarcie sekcji **Szczegóły** (Ctrl+E) w celu określenia źródła listy dostępnych do wyboru elementów.

The screenshot shows a software interface titled 'PARAMETRY'. It contains a table with columns: Lp, Nazwa, Etykieta, and Typ. The first row has values: 1, grupa, Grupa, and Lista bazodanowa. Below the table is a detailed configuration window for a database list parameter. The window has buttons for 'OK' and 'Anuluj'. The title is 'SZCZEGÓŁY PARAMETRU TYPU LISTA BAZODANOWA'. It contains four fields: 'Główny element' with value 'GrupaAsortymentu', 'Zapytanie' with value 'source', 'Pokazywane pole' with value 'Nazwa', and 'Początkowy wybór' with value 'Podstawowa'. Each field has a dropdown arrow on its right.

rys. 9. Szczegóły parametru listowego bazodanowego.

Definiowanie parametru typu lista bazodanowa wymaga wskazania źródła danych, podobnie jak dla całego zestawienia, poprzez określenie głównego elementu oraz zapytania z jego użyciem. W większości przypadków zapytania nie trzeba modyfikować. Należy za to określić właściwość (**Pokazywane pole**), która ma być widoczna na liście dostępnych wartości i wybrać z listy **Początkowy wybór** definiowanego parametru.

Korzystając z parametru bazodanowego, np. przy definiowaniu warunku **Where**, należy posługiwać się wartościami o typach prostych, czyli nie porównywać z elementami listy, a jedynie z ich identyfikatorami lub innymi właściwościami. Poniższy przykład pokazuje jak to wykonać z wykorzystaniem grup asortymentu.

Przykład raportu własnego z parametrem listowym bazodanowym:

PODSTAWOWE

Nazwa:	Asortyment z grupy	Widoczny w:	☒ Subiekt
Główny element:	Asortyment		☒ Rachmistrz
Opis:	Pokazuje nazwy asortymentu z wybranej grupy		☒ Rewizor
Grupa:	Asortyment		☒ Gratyfikant
			☒ Gestor
			☒ Biuro

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	grupa	Grupa	Lista bazodanowa

ŹRÓDŁO DANYCH

Zapytanie:

```

source
    .Where(a => a.Grupa.Id == grupa.Id)
    .Select(aso => new Wynik
    {
        Id = aso.Id,
        Nazwa = aso.Nazwa,
    });

```

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

KOLUMNY

Lp	Tytuł	Pole	Typ	Pole precyzji	Pole sufiksu	Tekst dla braku wartości	S
1	Nazwa	Nazwa	Tekst				☐

W przypadku parametru listowego bazodanowego nie ma znacznika określającego obsługę braku wartości dla tego parametru. Jednakże odpowiednie skonstruowanie zapytania przy definiowaniu szczegółów parametru bazodanowego pozwala na dodanie do listy wyboru specjalnych wartości takich jak np. *(dowolny)* czy *(brak)*. Oczywiście należy je potem odpowiednio uwzględnić w zapytaniu definiującym raport. Poniższy przykład pokazuje jak to zrobić dla grup asortymentu. Najpierw definiujemy szczegóły parametru dodając te specjalne elementy:

Lp	Nazwa	Etykieta	Typ
1	grupa	Grupa	Lista bazodanowa

☒ OK
 ☐ Anuluj

SZCZEGÓŁY PARAMETRU TYPU LISTA BAZODANOWA

Główny element: GrupaAsortymentu

Zapytanie:


```
source.ToList().Concat(
    new GrupaAsortymentu[] {
        new GrupaAsortymentu { Id = -1, Nazwa = "(brak)" },
        new GrupaAsortymentu { Id = -1, Nazwa = "(dowolna)" },
    })
.OrderBy(grupa => grupa.Nazwa)
```

Pokazywane pole: Nazwa

Początkowy wybór: Podstawowa

rys. 10. Szczegóły parametru listowego bazodanowego z dodanymi elementami specjalnymi.

Następnie poprawiamy zapytanie w raporcie tak, aby uwzględniało dodane specjalne wartości parametru *grupa*. W tym przypadku dla wartości *(dowolny)* zaprezentowane zostaną asortymenty z każdej grupy, natomiast wybór wartości *(brak)* spowoduje pojawienie się jedynie asortymentu nie przypisanego do żadnej grupy towarowej.

ŹRÓDŁO DANYCH

Zapytanie:


```
source
    .Where(a => grupa.Id == -2 ||
        (grupa.Id == -1 && a.Grupa == null) ||
        a.Grupa.Id == grupa.Id)
    .Select(aso => new Wynik
    {
        Id = aso.Id,
        Nazwa = aso.Nazwa,
    });
```

Parametry globalne

W zapytaniu (sekcja **Źródło danych**) można odwoływać się do parametrów globalnych, wynikających z kontekstu programu: bieżącego oddziału, magazynu, okresu obrachunkowego itd. Pozwalają one dopasować wyniki raportu do aktualnych parametrów pracy. Do parametrów globalnych należy odwoływać się z użyciem słowa **globalne**.

Lista dostępnych parametrów globalnych:

- ***globalne.Oddzial***, ***globalne.OddzialId*** - oddział, identyfikator oddziału
- ***globalne.Magazyn***, ***globalne.MagazynId*** - magazyn, identyfikator magazynu
- ***globalne.RachunekBankowy***, ***globalne.RachunekBankowyId*** - rachunek bankowy, identyfikator rachunku bankowego
- ***globalne.StanowiskoKasowe***, ***globalne.StanowiskoKasowyId*** - stanowisko kasowe, identyfikator stanowiska kasowego
- ***globalne.Uzytkownik***, ***globalne.UzytkownikId*** - użytkownik, identyfikator użytkownika
- ***globalne.Pracownik***, ***globalne.PracownikId*** - pracownik, identyfikator pracownika
- ***globalne.DrukarkaFiskalna***, ***globalne.DrukarkaFiskalnaId*** - drukarka fiskalna, identyfikator drukarki
- ***globalne.Waluta***, ***globalne.WalutaId*** - waluta systemowa, identyfikator waluty systemowej
- ***globalne.OkresObrachunkowy***, ***globalne.OkresObrachunkowyId***, ***globalne.OkresObrachunkowyOd***, ***globalne.OkresObrachunkowyDo*** - okres obrachunkowy, identyfikator okresu obrachunkowego, data rozpoczęcia, data zakończenia
- ***globalne.DataBlokady*** - data blokady
- ***globalne.DataAplikacyjna*** - data aplikacyjna

Dodawanie raportu własnego - sekcja ŹRÓDŁO DANYCH

Sekcja ta stanowi kluczowy element raportu własnego LINQ. Określa ona dane, które będą prezentowane w raporcie oraz sposób wykorzystania parametrów.

ŹRÓDŁO DANYCH

Zapytanie:

```
source
    .Where(a => a.Nazwa.StartsWith(poczatekNazwy))
    .Select(aso => new Wynik
    {
        Id = aso.Id,
        Nazwa = aso.Nazwa,
    });
```

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

rys. 11. Sekcja Źródło danych formularza edycji raportu własnego LINQ.

Pole **Zapytanie** definiuje dane pobierane z bazy na potrzeby raportu. Jest ono odpowiednikiem zapytania SQL wykorzystywanego do definiowania raportów w systemie

InsERT GT. Zapytania są tworzone z wykorzystaniem mechanizmu LINQ to Entities (<http://msdn.microsoft.com/en-us/library/bb399367.aspx>). W ramach zapytania można odwoływać się do wcześniej zdefiniowanych parametrów, wpisując ich nazwę (jak na rys. 11.) lub do parametrów globalnych. Element **source** stanowi predefiniowane słowo stanowiące zbiór wszystkich elementów wskazywanych przez główny element raportu (sekcja **Podstawowe**). Z punktu widzenia LINQ, jest to zmienna typu IQueryable, na której można przeprowadzać operacje typowe dla LINQ (jak Where czy Select). Jedną z opcji menu kontekstowego jest możliwość wstawienia przykładowego zapytania. Wybranie tej opcji spowoduje jednak zastąpienie aktualnej wartości przykładową. Przykładowe zapytanie zostanie również wstawione jako reakcja na wybór głównego elementu raportu w sekcji **Podstawowe** – jednak tylko wtedy, gdy pole nie zawiera aktualnie żadnej wartości.

Docelową zwracaną strukturę danych definiuje się przez projekcję. Ogranicza się dane pobierane z bazy do danych faktycznie wyświetlanych na raporcie. Projekcja jest wyrażeniem lambda (<http://msdn.microsoft.com/en-us/library/vstudio/bb397687.aspx>) tworzącym z pojedynczej encji pobranej z bazy danych typ **Wynik** (kompilowany podczas uruchamiania zapytania). Klasa **Wynik** jest obowiązkowa, jak na zaprezentowanym przykładzie na rys. 11.. W zapytaniu klasa **Wynik** musi zawierać jedno pole o nazwie **Id** oznaczające identyfikator obiektu. Przypomina o tym uwaga umieszczona pod polem przeznaczonym na wprowadzenie zapytania. Pola **Id** nie będzie można pokazywać w wyniku zapytania, jest ono zarezerwowane dla potrzeb operacyjności na obiektach. Gdyby jednak użytkownik chciał pokazywać identyfikator obiektu w wyniku swojego zapytania, wystarczy by powielił tę linijkę i nadał nową nazwę pola np. **Id2** (oznacza to, że klasa **Wynik** będzie miała dwie właściwości odwołujące się do tego samego pola - na rys. 11. powyżej, do pola **aso.Id**).

Do definiowania zapytania zalecane jest użycie narzędzia LINQPad (<http://www.linqpad.net/>) opisane w *rozdziale Testowanie raportów własnych w LINQPadzie*.

Parametry globalne

W zapytaniu można odwołać się do parametrów globalnych, wynikających z kontekstu programu: bieżącego oddziału, magazynu, okresu obrachunkowego itd. Do parametrów globalnych należy odwoływać się z użyciem słowa **globalne**. Lista dostępnych parametrów globalnych:

- **globalne.Oddzial**, **globalne.OddzialId** - oddział, identyfikator oddziału
- **globalne.Magazyn**, **globalne.MagazynId** - magazyn, identyfikator magazynu
- **globalne.RachunekBankowy**, **globalne.RachunekBankowyId** - rachunek bankowy, identyfikator rachunku bankowego

- **globalne.StanowiskoKasowe, globalne.StanowiskoKasoweId** - stanowisko kasowe, identyfikator stanowiska kasowego
- **globalne.Uzytkownik, globalne.UzytkownikId** - użytkownik, identyfikator użytkownika
- **globalne.Pracownik, globalne.PracownikId** - pracownik, identyfikator pracownika
- **globalne.DrukarkaFiskalna, globalne.DrukarkaFiskalnaId** - drukarka fiskalna, identyfikator drukarki
- **globalne.Waluta, globalne.WalutaId** - waluta systemowa, identyfikator waluty systemowej
- **globalne.OkresObrachunkowy, globalne.OkresObrachunkowyId, globalne.OkresObrachunkowyOd, globalne.OkresObrachunkowyDo** - okres obrachunkowy, identyfikator okresu obrachunkowego, data rozpoczęcia, data zakończenia
- **globalne.DataBlokady** - data blokady
- **globalne.DataAplikacyjna** - data aplikacyjna

Standardowe (proste) pola własne

W zapytaniu można odwołać się do wartości standardowych (prostych) pól własnych. Każde standardowe pole własne dostępne jest po przez właściwość **obj.PolaWlasne.PoleWlasneX**, gdzie **obj** to obiekt np. asortyment, a **X** to numer od 1 do 8. Przykładowo, aby odwołać się do 3-go standardowego pola własnego z asortymentu należy użyć następującego zapytania LINQ:

```
source
    .Select(aso => new Wynik
    {
        Id = aso.Id,
        Nazwa = aso.Nazwa,
        PoleProste1 = aso.PolaWlasne.PoleWlasne3
    });
```

Zaawansowane pola własne w wersji 2

W zapytaniu można odwołać się do wartości zaawansowanych pól własnych w wersji 2. Przykładowo, jeżeli w asortymencie zdefiniowane są następujące pola własne w wersji 2.:

Zaawansowane:

Lp	Id	Nazwa	Typ	Nazwa słownika	Min. liczba widocznych linii	Maks. liczba widocznych linii	Precyzja	Wartość domyślna
1	S0	pz tekst	Tekst	(brak)	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
2	L0	pz długi tekst	Długi tekst	(brak)	2	50	(nie dotyczy)	(brak)
3	I0	pz int	Liczba całkowita	(brak)	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
4	D0	pz decimal	Liczba rzeczywista	(brak)	(nie dotyczy)	(nie dotyczy)	2	(brak)
5	B0	pz bool	Wartość logiczna	(brak)	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	☐
6	T0	pz data	Data	(brak)	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
7	I1	pz słownik	Słownik własny	Kolory	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
8	I2	pz słownik sql o kluczu Int	Słownik własny SQL	Słownik sql int	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
9	G0	pz słownik sql o kluczu Guid	Słownik własny SQL	Słownik sql guid	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
10	G1	pz waluta	Słownik systemowy	Waluty	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
11	I3	pz magazyn	Słownik systemowy	Magazyny	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
12	I4	pz rachunek	Słownik systemowy	Rachunki bankowe	(nie dotyczy)	(nie dotyczy)	(nie dotyczy)	(brak)
		Nazwa						

rys. 12. Przykładowe pola zaawansowane zdefiniowane dla asortymentu

wtedy zapytanie odczytujące wartości powyższych pól wygląda tak:

```
null; //ta linijka jest wymagana na początku
```

```
//pomocnicze instrukcje służące do odczytu pozycji słowników
var kolory = UtworzZapytanieOElementySłownikaWłasnego("Kolory");
var słownikSqlInt = UtworzZapytanieOElementySłownikaWłasnegoSql<int>("Słownik sql int");
var słownikSqlGuid = UtworzZapytanieOElementySłownikaWłasnegoSql<Guid>("Słownik sql guid");
var waluty = UtworzZapytanieOElementySłownikaWalut();
var magazyny = UtworzZapytanieOElementySłownikaMagazynow();
var rachunki = UtworzZapytanieOElementySłownikaRachunkowBankowych();

//właściwe zapytanie do źródła danych
result = source.Select(aso => new Wynik
{
    Id = aso.Id,
    Nazwa = aso.Nazwa,
    PzTekst = aso.PolaWlasneAdv2.Get<string>("pz tekst"),
    PzDługiTekst = aso.PolaWlasneAdv2.Get<string>("pz długi tekst"),
    PzInt = aso.PolaWlasneAdv2.Get<int?>("pz int"),
    PzDecimal = aso.PolaWlasneAdv2.Get<decimal?>("pz decimal"),
    PzBool = aso.PolaWlasneAdv2.Get<bool?>("pz bool"),
    PzData = aso.PolaWlasneAdv2.Get<DateTime?>("pz data"),

    PzSłownik = kolory.FirstOrDefault(p => p.Id == aso.PolaWlasneAdv2.Get<int?>("pz słownik")).Wartosc,

    PzSłownikSqlInt = słownikSqlInt.FirstOrDefault(p => p.Klucz == aso.PolaWlasneAdv2.Get<int?>("pz słownik sql o kluczu Int")).Wartosc,

    PzSłownikSqlGuid = słownikSqlGuid.FirstOrDefault(p => p.Klucz == aso.PolaWlasneAdv2.Get<Guid?>("pz słownik sql o kluczu Guid")).Wartosc,

    PzWaluta = waluty.Select(w => new { w.Id, Wartosc = w.Symbol + " " + w.Nazwa }).FirstOrDefault(w => w.Id == aso.PolaWlasneAdv2.Get<Guid?>("pz waluta")).Wartosc,

    PzMagazyn = magazyny.Select(m => new { m.Id, Wartosc = m.Symbol + " " + m.Nazwa }).FirstOrDefault(m => m.Id == aso.PolaWlasneAdv2.Get<int?>("pz magazyn")).Wartosc,

    PzRachunek = rachunki.Select(r => new { r.Id, Wartosc = r.Nazwa }).FirstOrDefault(r => r.Id == aso.PolaWlasneAdv2.Get<int?>("pz rachunek")).Wartosc
});
```

Poniżej to samo zapytanie widoczne w sekcji „ŹRÓDŁO DANYCH”:

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

```
null; //ta linijka jest wymagana na początku

//pomocnicze instrukcje służące do odczytu pozycji słowników
var kolory = UtworzZapytanieOElementySlovníkaWlasnego("Kolory");
var słownikSqlInt = UtworzZapytanieOElementySlovníkaWlasnegoSql<int>("Słownik sql int");
var słownikSqlGuid = UtworzZapytanieOElementySlovníkaWlasnegoSql<Guid>("Słownik sql guid");
var waluty = UtworzZapytanieOElementySlovníkaWalut();
var magazyny = UtworzZapytanieOElementySlovníkaMagazynow();
var rachunki = UtworzZapytanieOElementySlovníkaRachunkowBankowych();

//właściwe zapytanie do źródła danych
result = source.Select(aso => new Wynik
{
    Id = aso.Id,
    Nazwa = aso.Nazwa,
    PzTekst = aso.PolaWlasneAdv2.Get<string>("pz tekst"),
    PzDlugiTekst = aso.PolaWlasneAdv2.Get<string>("pz dlugi tekst"),
    PzInt = aso.PolaWlasneAdv2.Get<int>("pz int"),
    PzDecimal = aso.PolaWlasneAdv2.Get<decimal>("pz decimal"),
    PzBool = aso.PolaWlasneAdv2.Get<bool>("pz bool"),
    PzData = aso.PolaWlasneAdv2.Get<DateTime>("pz data"),
    PzSlovník = kolory.FirstOrDefault(p => p.Id == aso.PolaWlasneAdv2.Get<int>("pz słownik")).Wartosc,
    PzSlovníkSqlInt = słownikSqlInt.FirstOrDefault(p => p.Klucz == aso.PolaWlasneAdv2.Get<int>("pz słownik sql o kluczu Int")).Wartosc,
    PzSlovníkSqlGuid = słownikSqlGuid.FirstOrDefault(p => p.Klucz == aso.PolaWlasneAdv2.Get<Guid>("pz słownik sql o kluczu Guid")).Wartosc,
    PzWaluta = waluty.Select(w => new { w.Id, Wartosc = w.Symbol + " " + w.Nazwa }).FirstOrDefault(w => w.Id == aso.PolaWlasneAdv2.Get<Guid>("pz waluta")).Wartosc,
    PzMagazyn = magazyny.Select(m => new { m.Id, Wartosc = m.Symbol + " " + m.Nazwa }).FirstOrDefault(m => m.Id == aso.PolaWlasneAdv2.Get<int>("pz magazyn")).Wartosc,
    PzRachunek = rachunki.Select(r => new { r.Id, Wartosc = r.Nazwa }).FirstOrDefault(r => r.Id == aso.PolaWlasneAdv2.Get<int>("pz rachunek")).Wartosc
});
```

rys. 13. Zapytanie odwołujące się do zaawansowanych pól własnych, zdefiniowanych dla asortymentu

W powyższym zapytaniu wykorzystano następujące lokalnie zdefiniowane metody C#:

- `UtworzZapytanieOElementySlovníkaWlasnego(string nazwaSlovníkaWlasnego, bool filtrowane = true)` - tworzy zapytanie LINQ do źródła danych zwracające pozycje danego słownika własnego
 - Zwracany typ: `IQueryable<PozycjaSlovníkaWlasnego>`
- `UtworzZapytanieOElementySlovníkaWlasnegoSql<TKey>(string nazwaSlovníkaWlasnegoSql, bool filtrowane = true)` - tworzy zapytanie LINQ do źródła danych zwracające pozycje danego słownika własnego SQL, gdzie TKey to typ klucza (int lub Guid)
 - Zwracany typ: `IQueryable<ElementSlovníkowegoZrodlaDanych<TKey>>`
- `UtworzZapytanieOElementySlovníkaWalut()` - tworzy zapytanie LINQ do źródła danych zwracające waluty dostępne po przez słownik systemowy walut
 - Zwracany typ: `IQueryable<Waluta>`
- `UtworzZapytanieOElementySlovníkaMagazynow()` - tworzy zapytanie LINQ do źródła danych zwracające magazyny dostępne po przez słownik systemowy magazynów
 - Zwracany typ: `IQueryable<Magazyn>`
- `UtworzZapytanieOElementySlovníkaRachunkowBankowych()` - tworzy zapytanie LINQ do źródła danych zwracające rachunki dostępne po przez słownik systemowy rachunków bankowych
 - Zwracany typ: `IQueryable<RachunekBankowy>`

Dodawanie raportu własnego - sekcja KOLUMNY

Sekcja ta służy do zdefiniowania kolumn raportu własnego. Umożliwia ustawienie szczegółowych właściwości tych kolumn.

KOLUMNY							
SYNCHRONIZUJ KOLUMNY							
Lp	Tytuł	Pole	Typ	Pole precyzji	Pole sufiksu	Tekst dla braku wartości	S
1	Nazwa	Nazwa	Tekst	(brak)	(brak)		☐

rys. 14. Sekcja Kolumny formularza edycji raportu własnego LINQ.

Dostępne właściwości definiowanej kolumny:

- **Tytuł** - nagłówek kolumny w raporcie własnym widoczne po jego uruchomieniu.
- **Pole** - nazwa *pola z projekcji*, której wartość będzie wyświetlona w danej kolumnie.
- **Typ** - rodzaj danych wyświetlanych w kolumnie. Obecnie dostępne są kolumny tekstowe, liczbowe, datowe oraz logiczne. Wybór rodzaju kolumny wpływa również na dodatkowe opcje kolumny dostępne poprzez polecenie **Szczegóły** (Ctrl+E) w menu kontekstowym.
- **Pole precyzji, pole sufiksu** - dotyczą tylko kolumn liczbowych. Precyzują one *nazwy pól projekcji*, które określają precyzję (liczbę cyfr dziesiętnych) oraz sufix (tekst wyświetlany po liczbie, np. nazwa jednostki miary) dla wartości kolumny.
- **Tekst dla braku** - dotyczy tylko kolumn datowych – określa ono wartość, jaka ma pojawić się w polu, jeżeli ma ono wartość pustą (np. „brak”).
- **Sortowanie** (skrót **S**) - umożliwia wybór kolumny, po której domyślnie następuje sortowanie danych w raporcie własnym. Możliwe jest wybranie tylko jednej takiej kolumny.

Kolumny prezentowane są w kolejności ich wprowadzenia. W celu zmiany kolejności kolumn można wykorzystać polecenia menu kontekstowego lub odpowiednie skróty klawiszowe (podobnie jak w sekcji **Parametry**).

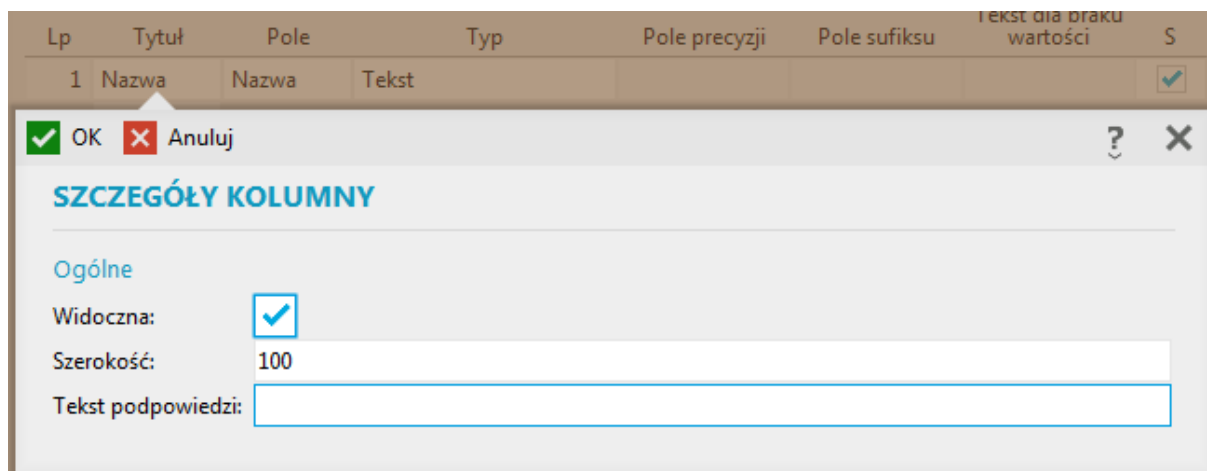
Dla wszystkich rodzajów kolumn można określić ich domyślną widoczność w raporcie (ustawienie to może być zmienione z poziomu konfiguracji listy w raporcie), domyślną szerokość kolumny (w pikselach) oraz tekst podpowiedzi widoczny po zatrzymaniu kursora myszy nad nagłówkiem kolumny.

Wymaganiem jest by wszystkie (oprócz **Id**) pola zdefiniowane w klasie **Wynik**, zostały zdefiniowane w sekcji **Kolumny**. Jeśli tak się nie stanie, wystąpi błąd kompilacji zapytania, gdyż nie będzie możliwe określenie typów pól klasy **Wynik** (w sekcji kolumny definiowany jest właśnie m.in. typ danych dla pól – opisane powyżej).

W procesie definiowania kolumn pomocny jest przycisk SYNCHRONIZUJ KOLUMNY. Jego zadanie polega na wypełnieniu tabeli z definicją kolumn na podstawie wprowadzonego zapytania. Po naciśnięciu tego przyciski dodawane są kolumny dla pól jeszcze nie zmapowanych, usuwane kolumny odpowiadające polom już usuniętym oraz synchronizowanie typy dla pól, którym zmienił się typ (np. z Tekst na Data i czas)

Kolumna tekstowa

Kolumna tekstowa umożliwia ustawienie tylko standardowych parametrów, wspólnych dla wszystkich rodzajów kolumn.

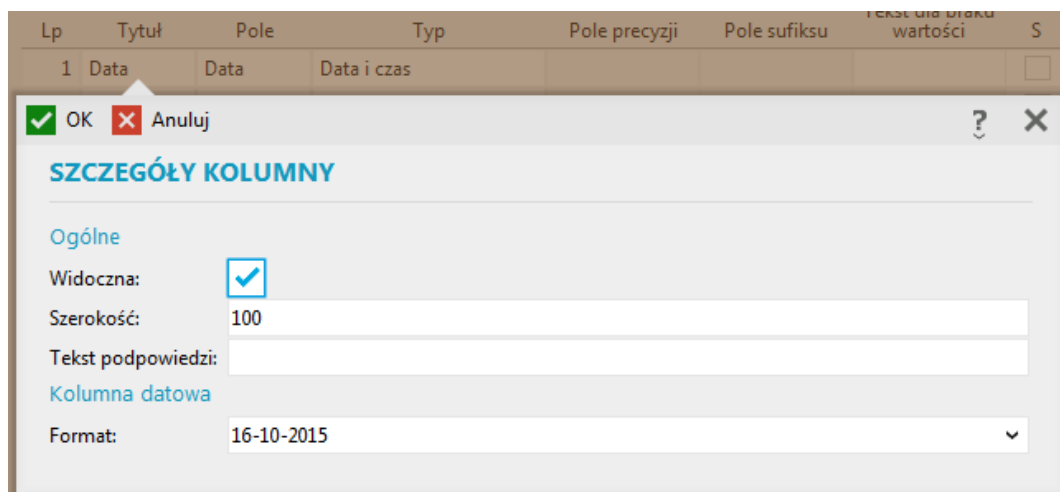


The screenshot shows a table with columns: Lp, Tytuł, Pole, Typ, Pole precyzji, Pole sufiksu, Tekst dla braku wartości, and S. The first row is selected, showing '1', 'Nazwa', 'Nazwa', 'Tekst'. Below the table is a dialog box titled 'SZCZEGÓŁY KOLUMNY'. It has 'OK' and 'Anuluj' buttons. Under the 'Ogólne' section, there are three settings: 'Widoczna:' with a checked checkbox, 'Szerokość:' with a text box containing '100', and 'Tekst podpowiedzi:' with an empty text box.

rys. 15. Szczegóły kolumny tekstowej.

Kolumna datowa

Poza standardowymi ustawieniami wspólnymi dla wszystkich rodzajów kolumn kolumna datowa umożliwia zdefiniowanie własnego formatu wyświetlania daty i czasu (definiowanie formatu daty i czasu opisane w <http://msdn.microsoft.com/en-us/library/8kb3ddd4.aspx>).



The screenshot shows the same table as in Figure 15, but the first row is '1', 'Data', 'Data', 'Data i czas'. The dialog box 'SZCZEGÓŁY KOLUMNY' is open. It has 'OK' and 'Anuluj' buttons. Under the 'Ogólne' section, there are three settings: 'Widoczna:' with a checked checkbox, 'Szerokość:' with a text box containing '100', and 'Tekst podpowiedzi:' with an empty text box. Below this, under the 'Kolumna datowa' section, there is a 'Format:' label and a dropdown menu showing '16-10-2015'.

rys. 16. Szczegóły kolumny datowej.

Kolumna liczbowa

Poza standardowymi ustawieniami, wspólnymi dla wszystkich rodzajów kolumn, w przypadku kolumny liczbowej można określić również precyzję oraz sufix. Ustawienia te różnią się od ustawień dostępnych na liście kolumn – na liście specyfikowane jest pole, co umożliwia uzależnienie precyzji czy sufiksu od danych zwróconych przez bazę danych,

natomiast pola w sekcji **Szczegóły** umożliwiają ustawienie stałej precyzji i sufiksu dla wszystkich pól w kolumnie – niezależnie od tego, jakie wyniki zwróci baza danych.

OK Anuluj

SZCZEGÓŁY KOLUMNY

Ogólne

Widoczna: ☒

Szerokość: 100

Tekst podpowiedzi:

Kolumna liczbowa

Precyzja:

Sufiks:

rys. 17. Szczegóły kolumny liczbowej.

Kolumna logiczna

Poza standardowymi ustawieniami kolumna ta umożliwia zdefiniowanie tekstu wyświetlanego w przypadku, gdy wartością kolumny jest **prawda** oraz tekstu dla wartości **fałsz**.

OK Anuluj

SZCZEGÓŁY KOLUMNY

Ogólne

Widoczna: ☒

Szerokość: 100

Tekst podpowiedzi:

Kolumna logiczna

Tekst dla wartości PRAWDA: tak

Tekst dla wartości FAŁSZ: nie

rys. 18. Szczegóły kolumny logicznej.

Dodawanie raportu własnego - sekcja PODGLĄD

Sekcja umożliwia bieżące weryfikowanie wyglądu i wartości raportu będącego wynikiem wprowadzonych ustawień. W celu wygenerowania lub zaktualizowania podglądu należy wcisnąć przycisk **Testuj zapytanie**.

PODGLĄD	
TESTUJ ZAPYTANIE	
Nazwa	
Balsam do ciała nawilżający 200 ml	
Balsam do ciała ultranawilżenie i dotlenienie 250	
Balsam do ciała intensywnie regenerujący 200 ml	
Dostawa do klienta	
Forever dezodorant 100ml	
So dezodorant perfumowany 100ml	
So dezodorant perfumowany 200ml	

rys. 19. Sekcja Podglądu formularza edycji raportu własnego LINQ

Jeżeli pomimo wciśnięcia przycisku **Testuj zapytanie** podgląd nie pojawia się, powodem może być błąd popełniony podczas definiowania parametrów, zapytania, bądź błędna definicja kolumn (złe typy kolumn dla wyświetlanych danych). Wymagany jest też by wszystkie (oprócz Id) pola zdefiniowane w klasie **Wynik**, zostały zdefiniowane w sekcji **Kolumny**. Informacje pojawiają się w postaci komunikatów lub błędów kompilacji. Błędy kompilacji wskazują miejsce w kodzie zapytania, gdzie występuje problem. Na rys. 17 pokazany jest błąd kompilacji wynikający z braku definicji kolumny dla pola Nazwa. W tym przypadku wystarczy skorzystać z opcji **Synchronizuj kolumny**, aby rozwiązać problem.

```
source
.Select((Waluta waluta) => new Wynik
{
    Id = waluta.Id,
    Symbol = waluta.Symbol,
    Nazwa = waluta.Nazwa,
});
```

Linia: 6
Znak: 9

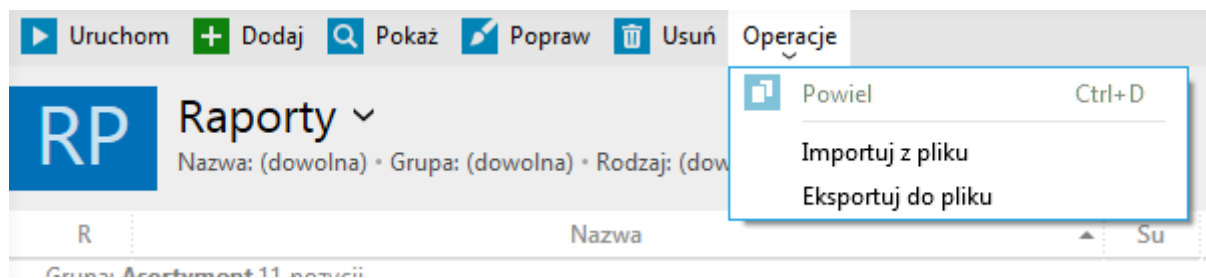
• NIE MOŻNA SKOMPILOWAĆ RAPORTU WŁASNEGO Z POWODU:
[LINIA: 6] [ZNAK: 9] [NUMER BŁĘDU: CS0117] W ELEMENTIE
„INSERT.MORIA.RAPORTY.RAPORTYWŁASNE.RF00643ADB3D44C7384B6619E7E6D4D54.WYNIK22
” BRAK DEFINICJI „NAZWA”.

rys. 20. Błąd kompilacji zapytania – brak definicji kolumny dla pola Nazwa.

Eksportowanie i importowanie

Raporty własne LINQ, po dodaniu i zapisaniu można wyeksportować do pliku XML. Jest to dogodny sposób na ich przenoszenie pomiędzy różnymi wdrożeniami.

Importu i eksportu raportu można dokonać zarówno z serwisu (otworzy się formatka tworzenia nowego raportu własnego) jak i bezpośrednio w menu górnym formatki dodawania nowego raportu (należy wybrać **Dodaj** w serwisie a następnie w otwartym oknie operację **Importuj...**). Aby zaimportować raport własny LINQ użytkownik musi mieć uprawnienie do edycji raportów własnych. Jest to to samo uprawnienie, które wymagane jest do dodawania nowego raportu.



rys. 21. Operacja eksportu i importu raportu własnego, dostępne w serwisie Raportów.

Ze względu na sposób definiowania raportów własnych, nie ma dla nich funkcji powielania. Gdy zachodzi potrzeba stworzenia raportu, którego bazą będzie inny raport własny, należy wykonać kolejno eksport do pliku i import z nową nazwą, a następnie skonfigurowanie nowego raportu.

Uruchamianie raportów własnych LINQ oraz jego edycja i podgląd.

Wraz ze zdefiniowaniem raportu własnego dodawane jest w systemie uprawnienie do jego uruchamiania. Domyślnie jest ono nadawane roli Administrator, a użytkownicy przypisani do innych ról nie otrzymują tego uprawnienia, a więc nie mogą uruchomić nowego raportu. Uprawnienie można im nadać w module Konfiguracja -> Użytkownicy. Aby to zrobić, wybieramy na liście użytkownika, któremu chcemy nadać uprawnienie. Następnie po wejściu w edycję za pomocą opcji **Popraw**, po kliknięciu **Szczegółowa lista uprawnień** należy wyszukać nowe uprawnienie w kategorii **Raporty własne**. Będzie to wpis o takiej nazwie jak dodany raport własny. Zaznaczając znacznik nadajemy uprawnienie.

Proces uruchamiania raportów własnych jest taki sam jak w przypadku raportów standardowych. Należy w module Raporty, znaleźć na liście dwukrotnie kliknąć na raport, który chcemy uruchomić lub zaznaczyć na liście i kliknąć w górnym menu ikonkę Uruchom.

Po uruchomieniu raportu pojawia się widok prezentujący jego wyniki. Nad listą, w menu, dostępne są operacje wynikające z wybranego typu głównego elementu, takie jak w module odpowiadającym temu elementowi. Nad listą są też dostępne parametry wyspecyfikowane podczas definiowania raportu własnego. Za pomocą tych parametrów użytkownik raportu steruje zakresem analizowanych lub prezentowanych danych.

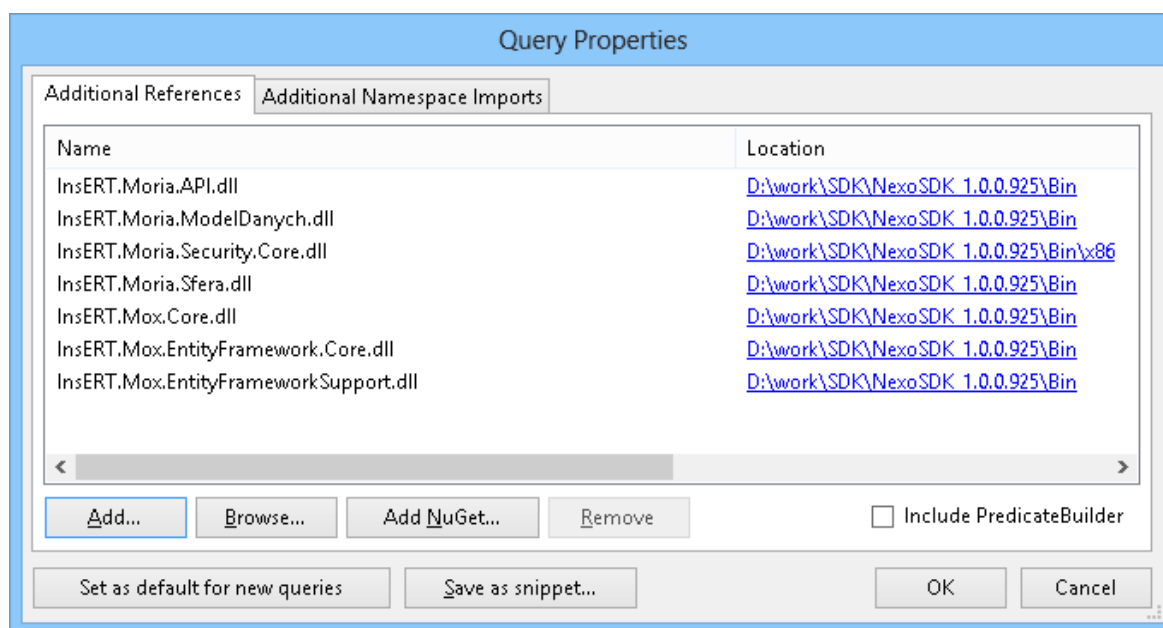
Modyfikacje raportu własnego, jego definicji wykonać można z poziomu listy raportów w module Raporty. W tym celu należy wybrać z menu opcję **Popraw**. Zmiany należy wprowadzać zgodnie z niniejszą instrukcją tworzenia raportów własnych LINQ.

Testowanie raportów własnych w LINQPadzie

Narzędzie LINQPad można pobrać ze strony <http://www.linqpad.net/>

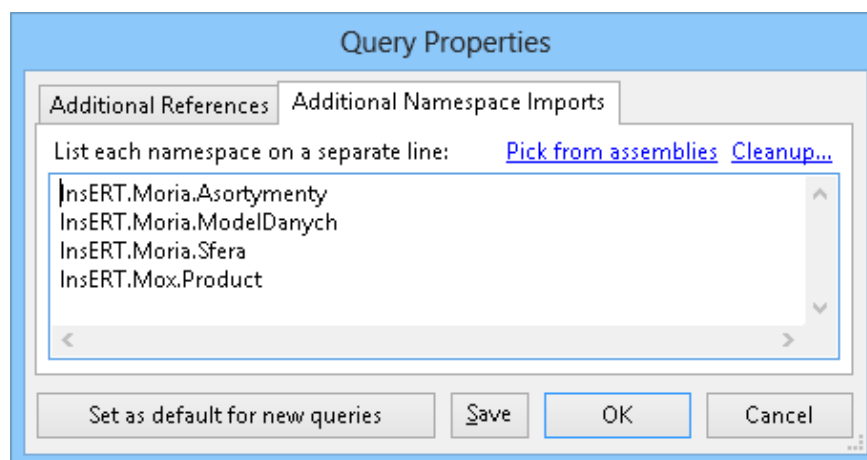
Nie należy korzystać z opcji Add connection/Entity Framework (lewy panel najlepiej zamknąć, bo nie będzie wykorzystywany). Można natomiast testować raporty własne, tworząc w LINQPadzie szkieletowe rozwiązania sferyczne. W edytorze zapytania należy wybrać **Language: C# Statements**. Zaznaczyć **Results to Data Grids** (skrót Ctrl+Shift+G, menu Query -> Results to Data Grids, bądź przycisk obok combo **Language**).

We właściwościach zapytania (F4, menu Query -> Query Properties) należy dodać referencje do bibliotek sferycznych zgodnych z wersją bazy danych, do której będziemy się podłączać.



rys. 22. Okno dodawania referencji w narzędziu LINQPad.

Należy również zdefiniować importowane przestrzenie nazw.



rys. 23. Okno definiowania przestrzeni nazw w narzędziu LINQPad.

W edytorze zapytania należy wkleić szkieletowe rozwiązanie:

```
var danePolaczenia = DanePolaczenia.Jawne("(local)", "nexo_Demo",
autentykacjaWindowsDoSerwera: true);
var nexo = new MenedzerPolaczen().Polacz(danePolaczenia, ProductId.Subiekt,
ProductId.Rachmistrz, ProductId.Rewizor);
nexo.ZalogujOperatora("demouser", "robocze");
var source = nexo.PodajObiektTypu<IASortymenty>().Dane.Wszystkie();
//zdefiniowane zmienne dla parametrów - opcjonalnie
var poczatekNazwy = "s";
// zawartość pola "Zapytanie" sekcji „Źródło danych” formularza raportu
własnego
source
.Where(a => a.Nazwa.StartsWith(poczatekNazwy))
.Select(
(aso) => new // należy usunąć nazwę klasy „Wynik” z formularza raportu
własnego
{
    Id = aso.Id,
    Nazwa = aso.Nazwa,
}
//koniec zawartości pola „Zapytanie”
)
.Dump();
```

...podając oczywiście parametry **do swojej bazy danych**. Nazwę menedżera z linii 4 (tutaj IASortymenty) odpowiadającą wyborowi z pola **Główny element** z formularza raportu własnego można znaleźć w pomocy Sfery dla InsERT nexo.

W LINQPad nie należy się posługiwać klasą **Wynik**. Należy, w zapytaniu definiując strukturę pokazywanych danych, posłużyć się typem anonimowym (<http://msdn.microsoft.com/en-us/library/vstudio/bb397696.aspx>).

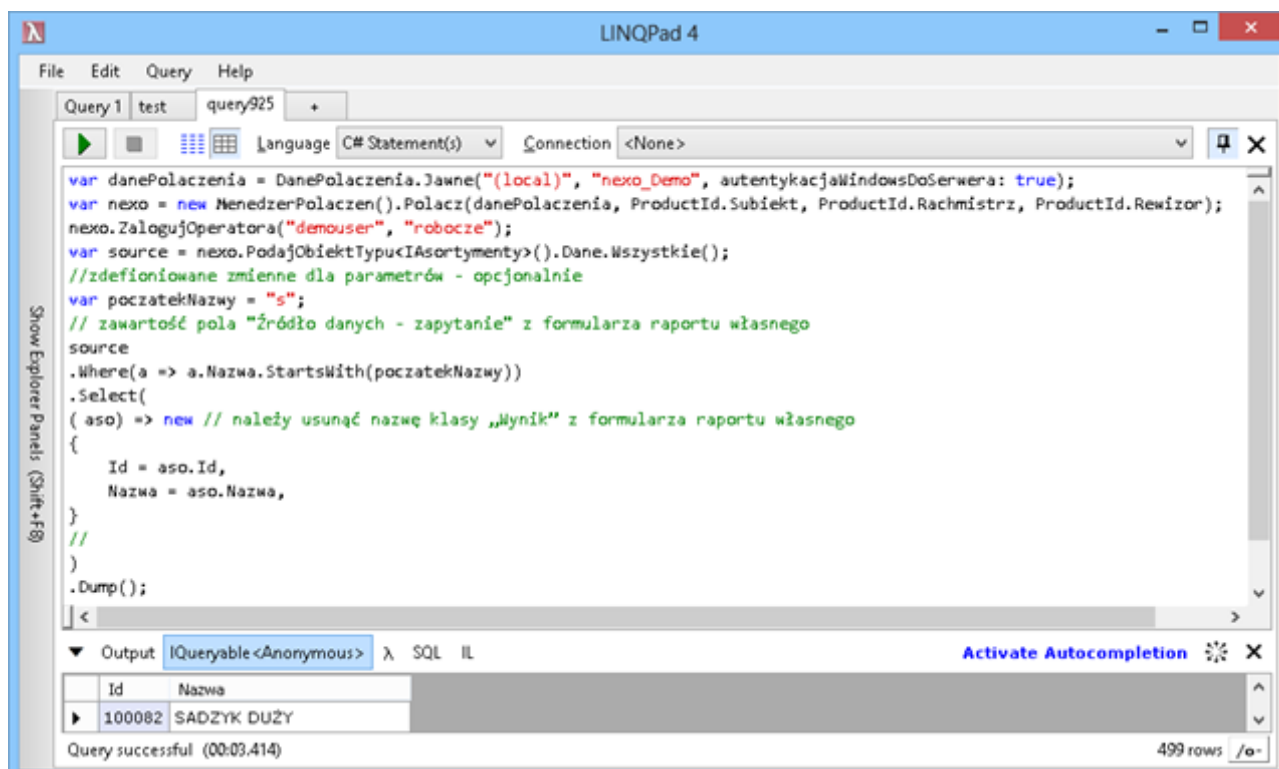
Dostępne pola danej encji można zobaczyć, kupując IntelliSense do LINQPada bądź w projekcji wybrać całą encję, tj.:

```
source
.Select(
(Asortyment aso) => aso)
```

lub w klauzuli Select użyć wyrażenia lambda .Select((obj) => obj) jeśli wcześniejsze operacje łączą już wiele encji ze sobą, a chcemy podejrzeć strukturę wszystkich danych, do których jest dostęp.

Podsumowując, poniżej zdefiniowane zapytanie w narzędziu LINQPad oraz widok raportu własnego opartego na tym samym zapytaniu w systemie InsERT nexo.

LINQPad:



rys. 24. Szkieletowe rozwiązanie sferyczne do raportu własnego w narzędziu LINQPad.

InsERT nexo:

PODSTAWOWE

Nazwa: Przykładowy raport LINQ asortymentu
Główny element: Asortyment
Opis: Przykładowy raport własny asortymentu w InsERT nexo
Grupa: Asortyment

Widoczny w:
☒ Subjekt
☒ Rachmistrz
☒ Rewizor
☒ Gratyfikant
☒ Gestor
☒ Biuro

PARAMETRY

Lp	Nazwa	Etykieta	Typ
1	poczatekNazwy	Pierwsza litera nazwy	Tekst

ŹRÓDŁO DANYCH

Zapytanie:

```

source
    .Where(a => a.Nazwa.StartsWith(poczatekNazwy))
    .Select(aso => new Wynik
    {
        Id = aso.Id,
        Nazwa = aso.Nazwa,
    });

```

Wynik zapytania musi posiadać pole o nazwie "Id", zawierające identyfikator obiektu.

rys. 25. Widok sekcji Podstawowe, Parametry i Źródło danych raportu własnego w InsERT nexo.